

Enhancement Two Narrative: Algorithms and Data Structures

Artifact overview

The artifact used for Enhancement Two was Travlr Getaways, a travel application built as part of my full-stack development course. The application has a full-featured front-end for customers to access their travel plans, an Express/Node.js backend API, database storage via MongoDB using Mongoose, and an Angular-based administrative single-page application to manage trip records.

When I completed this milestone, I again used the enhanced version of the original artifact, which was created during the software design and engineering enhancement. Enhancement Two added an algorithmic/data structure improvement to support recommending trips based on customer preferences via a new backend API call.

Rational

I chose Travlr Getaways for my e-portfolio because it is a complete full-stack app that can still benefit from many improvements. As compared to a small isolated class assignment, it had all the features that were needed to show me how to enhance an existing app in a systematic manner with, routing, API design, security/authentication, persistence, client-side interactions and testable server side behaviors.

In its original version, the majority of the trip operations available were based upon a simple CRUD model. Users and administrators could view lists of trips or find individual trips using their codes, however, there was no meaningful ability to discover trips. Because this feature limited my ability to use an algorithms/data structures enhancement for my artifact, it provided an excellent opportunity.

A travel application naturally lends itself well to utilizing search, filter, sort, and recommendation capabilities due to the fact that many times when travelers are selecting which trips they wish to take into account factors such as location, cost, and length of stay of the trip, as well as other trip specific characteristics. The enhanced artifact provides a new API endpoint called GET /api/trips/recommendations. This endpoint takes user input that include free form search criteria, maximum budget, maximum length of stay, and a result limit. The output will be a collection of recommended trips along with a numeric recommendation score and some human readable recommendation information.

This enhancement represents algorithmic thinking since it does much more than simply return items from the database. In addition to processing user's query criteria to create normalized data, build searchable tokens, select possible candidates for recommendations, eliminate based on constraint rules, rank the best match, determine the ordering of the final results based on some consistent rules.

Enhancement and Evidence

The primary implementation exists in the file `app_api/services/trip-recommendation-service.js`. Several steps with related data structures were implemented as part of this enhancement:

Step one: The service normalizes all of the input criteria. All prices such as \$900 and 900 are converted to number types. All durations such as 4 nights are converted to number type. Text fields are broken down into individual, lowercase search terms. Any invalid entries are excluded prior to ranking to ensure no misleading information is provided and to maintain predictability in how the API behaves.

Step Two: An inverted index is created using a JavaScript map where each key represents a searchable term (token) and each value represents a Set containing each trip's position. The use of an Inverted Index will allow the service to quickly determine potential trips based on token match instead of having to scan each searchable field for each potential rank during every ranking operation. While both approaches could provide quick performance for the current artifact size, using an Inverted Index will demonstrate scalability and make the trade-off explicit.

Step Three: Candidate trips are filtered against hard constraints. If a MaxPrice or MaxDuration has been entered, any Trips that fall outside of these boundaries will be removed from the Results.

Step Four: The service performs a weighted scoring model. Each trip receives points for matching search terms, budget fit, and duration fit. The scoring mechanism is intentionally simplistic and understandable. The returned response includes recommendation reasons to prevent the ranking process from being viewed as a black box.

Step Five: The service ranks the results based on the recommendations score. When two trips have identical scores, the lowest price is chosen. When both the score and price are equal, the original collection order is utilized as a stable tie-breaker. This provides deterministic output that is crucial for testing and user trust.

A new API route was added to `app_api/routes/index.js` above the existing `/trips/:tripCode` route. This ordering is significant since the routing performed by Express is sequential, therefore, the addition of the `/trips/recommendations` route first ensures that the word "recommendations" is treated as the recommendation endpoint instead of as another possible trip code.

New automated tests were added in `test/trip-recommendation-service.test.js` and the existing test suite for the trip service were modified to include additional verification of delegating services. Today's test suite includes verification for price parsing, duration parsing, inverted index creation, text-based searches, budget and duration filtering, weighted rankings, tie-breaking, validation errors, and services' boundaries.

Local verification successfully completed 19 out of 19 backend tests via `npm.cmd test`

Couse Outcome Alignment

The addition of this enhancement supports course outcome 3 by providing design and evaluation of computing solutions that utilize algorithmic principles and CS practices/standards to create a solution to a specific problem while weighing design constraints. Prior to this enhancement, there were no means to discover trips meaningfully. The solution utilized techniques such as tokenization, inverted indexing, sets, constraint filtering, weighted scoring, and stable sorting to rank recommended destinations.

This enhancement also supports course outcome 4 since the recommendation endpoint has added functionality to the travel application. Rather than being an academic algorithm positioned adjacent to the project, the recommendation endpoint is integrated into the current Express API and tested via the backend test suite. In doing so, the solution built upon the software design boundaries identified during enhancement one which made adding the algorithm as a separate service much simpler than inserting ranking logic directly within the controller.

Process, Challenges, and Leanings

The major takeaway from this enhancement has been that an application's algorithms need to relate to some type of actual problem a user may have. I could have added a stand-alone sort function or simple front-end filters. But doing so wouldn't have made the artifact better in a significant way. In my opinion the recommendation endpoint is more valuable because it relates to the travel domain and provides an expected feature on the application.

A major challenge I faced was determining what algorithm I could use to meet the milestones requirements, without over-complicating the artifact. The amount of training data needed for a full-fledged machine-learning based recommendation engine would be well beyond the scope of the data currently being used in this project. While a very simple database query-based filter would be easy to create, it would be far too simplistic to meet the requirements of the milestones. My chosen approach represents a compromise: using known CS constructs within the artifact, while keeping the implementations both explain-able, test-able and commensurate with the current state of development of the application.

Another challenge I faced was figuring out how to manage the issue of "imperfect" data. As you are aware, the Trip record table stores price and duration as string representations (\$799, 4 nights). Rather than modify the DB schema to accommodate these types of changes at this milestone, I chose to add parse logic which converts the string representation into numeric values prior to processing them for scoring purposes. Overall, I believe this was a good tradeoff. It allowed me to keep focus on the enhancements to the algorithms while allowing more significant schema-related issues to be addressed in future work related to the Database.

Finally, I came away from this enhancement understanding the importance of using deterministic tie breaking methods. Deterministic Tie Breaking means when two items have equal scores they will always appear in the same order. This is particularly relevant when testing recommendation logic. Prior to implementing deterministic tie breaking, I found it quite difficult to write adequate unit tests. For example, when writing unit tests, I never knew what order the items would come back in, even though they all had identical scores. Now that I've implemented deterministic tie breaking, I'm able to ensure that they come back in the correct order.